Dokumentacja backendowa CodeBin_WAT

Autor: Igor Filipiak

Baza danych

1. Typ Bazy Danych i Technologia

Nasz system opiera się na **Relacyjnej Bazie Danych (RDBMS)**. Komunikacja z bazą nie odbywa się poprzez surowe zapytania SQL, lecz przy użyciu wbudowanego w framework **Django ORM (Object-Relational Mapping)**.

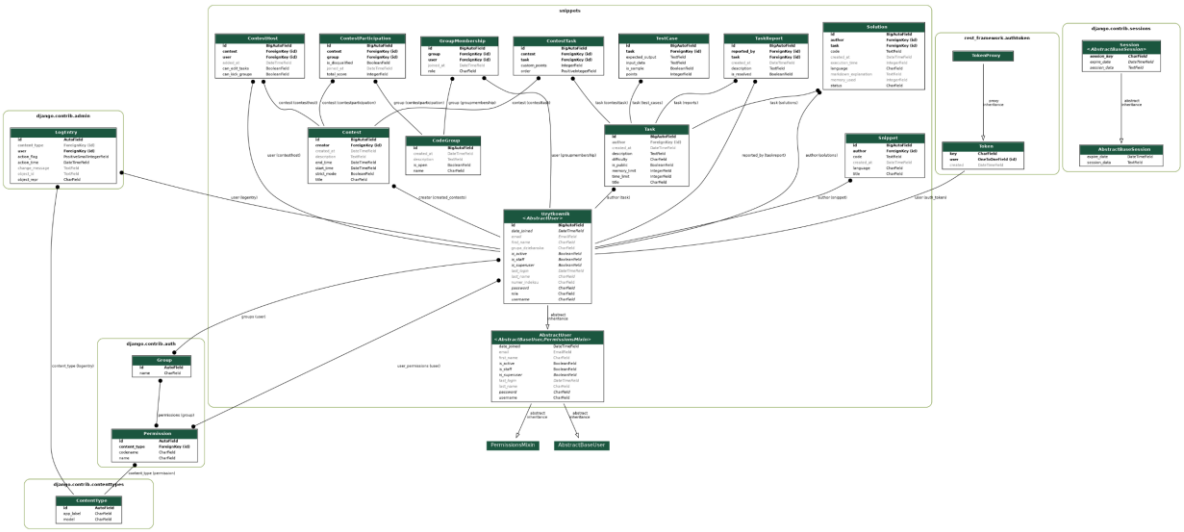
Dzięki takiemu podejściu struktura bazy jest definiowana jako klasy w języku Python (w pliku **models.py**), co pozwala na łatwe migracje i pełną niezależność od silnika bazy danych. Lokalnie system może korzystać z bazy SQLite, natomiast w środowisku kontenerowym (Docker) i produkcyjnym bez problemu przepina się na bardziej wydajne silniki (np. PostgreSQL), nie wymagając przy tym żadnych zmian w kodzie.

---

2. Struktura Tabel (Słownik Encji)

Baza danych została zaprojektowana w sposób modułowy. Poniżej znajduje się opis wszystkich głównych tabel oraz ich ról w systemie.

Zdjęcie bazy danych znajduje się również w pliku **diagram_bazy_danych.png**



Moduł Autoryzacji i Użytkowników

- **Uzytkownik (Rozszerzenie AbstractUser):** Centralny punkt systemu. Przechowuje podstawowe dane logowania (login, hasło, email) oraz dedykowane pola uczelniane: rola (Student/Profesor), numer_indeksu i grupa_dziekanska.

Moduł Zadań i Weryfikacji Kodu

- **Snippet:** Tabela pełniąca rolę notatnika. Przechowuje luźne kawałki kodu (code, language) tworzone przez użytkowników na "brudno", niezwiązane z żadnym konkretnym zadaniem.
- **Task (Zadanie):** Główna tabela z problemami algorytmicznymi. Zawiera tytuł, treść, poziom trudności oraz flagę is_public. Definiuje też krytyczne ograniczenia dla systemu sprawdzającego: time_limit (w ms) oraz memory_limit (w MB).
- **TestCase (Przypadek Testowy):** Zestawy danych służące do weryfikacji poprawności algorytmów. Każde zadanie posiada wiele testów (relacja Jeden-do-Wielu). Model zawiera dane wejściowe (input_data), oczekiwany wynik (expected_output) oraz flagę is_sample decydującą, czy test jest widoczny dla rozwiązującego, czy ukryty.
- **Solution (Rozwiązanie):** Łącznik pomiędzy Użytkownikiem a Zadaniem. Zapisuje wysłany kod źródłowy, język programowania oraz ewentualny opis (Markdown). Baza przechowuje tu również fizyczny czas wykonania i zużytą pamięć podczas ewaluacji oraz ostateczny status (np. Zaakceptowane, Błąd Kompilacji).

Moduł Społeczności

- **CodeGroup (Grupa):** Reprezentuje zrzeszenie użytkowników (np. konkretną klasę na uczelni). Definiuje nazwę, opis oraz flagę is_open, która określa, czy do grupy można dołączyć z "ulicy".
- **GroupMembership:** Tabela pośrednia (Many-to-Many) decydująca o tym, kto należy do jakiej grupy. Co ważne, przechowuje również role, pozwalając odróżnić zwykłego członka od Administratora danej grupy.

Moduł Turniejowy (Konkursy)

- **Contest (Konkurs):** Organizuje zadania w ograniczone czasowo ramy (start_time, end_time). Posiada flagę strict_mode, która aktywuje na frontendzie tryb zabezpieczeń antysciągowych (np. blokadę wyjścia z karty przeglądarki).
- **ContestParticipation:** Tabela pośrednia łącząca Konkurs z Grupami startującymi. Rejestruje łączny wynik (total_score) oraz status ewentualnej dyskwalifikacji (is_disqualified).

- **ContestHost:** Tabela delegująca uprawnienia. Pozwala Głównemu Twórcy konkursu przypisać innych użytkowników jako sędziów/pomocników z precyzyjnie określonymi prawami (np. możliwość modyfikacji zadań, wyrzucania grup).

Moduł Zgłoszeń

- **TaskReport (Zgłoszenie/Ticket):** Pozwala użytkownikom zgłaszać problemy z zadaniami (np. błędny test ukryty). Łączy konkretne zadanie z osobą zgłaszającą i udostępnia status rozwiązania problemu (is_resolved).

---

3. Pokrycie Wymagań Biznesowych (Historie Użytkowników - HU)

Architektura została ściśle dopasowana do wymagań projektowych. Tabela poniżej przedstawia, jak poszczególne modele realizują założenia HU:

- **Zarządzanie Zadaniami (HU 1.3, 1.4, 1.12):** Tabela Task pozwala na tworzenie zadań prywatnych i publicznych. Obsługę treści zapewnia elastyczne pole description (gotowe na parsowanie Markdown). Ograniczenia sprzętowe dla sprawdzarki kodu gwarantują pola time_limit oraz memory_limit.
- **System Ewaluacji Testowej (HU 1.7, 1.8):** Rozdzielenie testów jawnych od ukrytych zostało osiągnięte poprzez dedykowany model TestCase z flagą is_sample i opcją punktacji za każdy pojedynczy test.
- **Zarządzanie Społecznością (HU 2.1, 2.2):** Tabela CodeGroup oraz przypisana do niej relacja GroupMembership obsługuje zarówno zróżnicowanie dostępu (grupy otwarte/zamknięte), jak i hierarchię wewnątrz społeczności (role członka i admina).
- **Zabezpieczenia i Turnieje (HU 2.3):** System Konkursów (Contest) poprzez flagę strict_mode natywnie wspiera implementację mechanizmów antysciągowych, a zaawansowane tabele pośrednie umożliwiają organizację turniejów międzyszkolnych (wiele grup w jednym konkursie).
- **Utrzymanie Jakości (HU 1.10, 3.3):** Dedykowany model TaskReport implementuje pełnoprawny, śledzony system zgłaszania błędów przez studentów do wykładowców.

---

4. Wizualizacja i Aktualizacja Schematu

Zgodnie z zasadą DRY (Don't Repeat Yourself), nie utrzymujemy ręcznie rysowanych schematów bazy danych, ponieważ szybko stają się nieaktualne.

Wizualizacja bazy generowana jest **automatycznie** bezpośrednio z kodu źródłowego (models.py). Wykorzystujemy do tego połączenie programu systemowego **Graphviz** oraz biblioteki pythonowej **django-extensions**.

Instrukcja wygenerowania aktualnego diagramu (ERD): Aby pobrać najświeższy schemat relacji między tabelami, w głównym katalogu projektu uruchom poniższą komendę w terminalu:

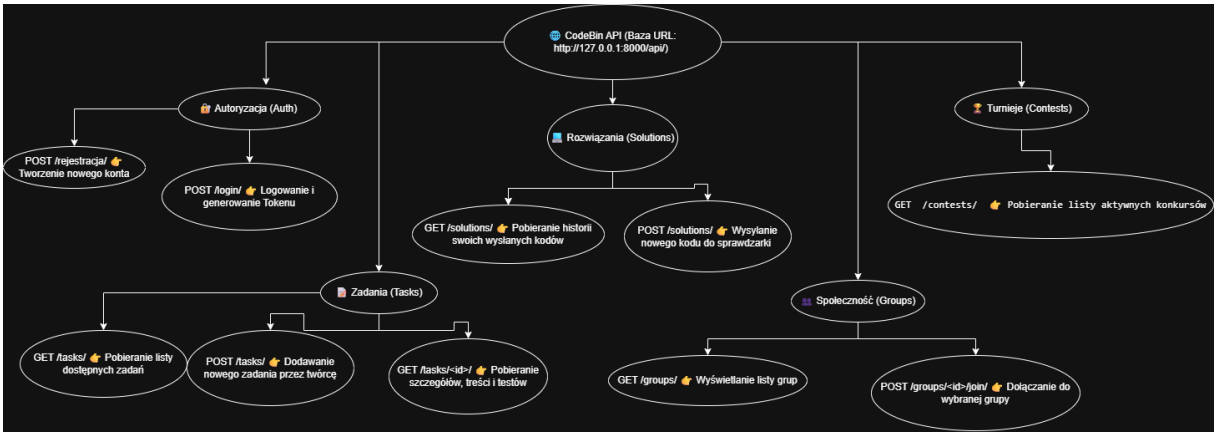
Bash

```
docker-compose exec web python manage.py graph_models -a -o
diagram_bazy_danych.png
```

Wygenerowany plik diagram_bazy_danych.png pojawi się w folderze roboczym i będzie zawierał wszystkie bieżące tabele, relacje (Klucze Obce) oraz typy danych.

Endpointy w API

Diagram endpoinów:



1. Zrealizowane Endpointy (Działające)

Posiadamy w pełni funkcjonalny moduł autoryzacji i uwierzytelniania oparty na tokenach (Token Authentication z Django REST Framework).

1.1 Rejestracja Użytkownika

- **Adres URL:** POST /api/rejestracja/
- **Kto ma dostęp:** Każdy (Brak kłódki / AllowAny)
- **Działanie z frontendem:** Frontend wysyła obiekt JSON z danymi z formularza. Backend waliduje dane (sprawdza unikalność maila, poprawność ról, wymogi uczelniane). Jeśli dane są błędne, backend zwraca status 400 Bad Request wraz z obiektem zawierającym tablice błędów dla konkretnych pól (co frontend teraz poprawnie łapie i wyświetla w czerwonej belce).
- **Oczekiwany Payload (JSON):**

JSON

```
{
  "username": "igor22",
  "email": "k12345@student.wat.edu.pl",
  "password": "SuperTajneHaslo123#",
  "first_name": "Igor",
  "last_name": "Filipiak",
  "rola": "student",
  "grupa_dziekanska": "WCY24KY2S1",
  "numer_indeksu": "11111"
}
```

- **Poprawna Odpowiedź:** Status 201 Created

1.2 Logowanie i Pobieranie Tokenu

- **Adres URL:** POST /api/logowanie/ (lub odpowiednik zdefiniowany w urls.py)
- **Kto ma dostęp:** Każdy
- **Działanie z frontendem:** Frontend wysyła poświadczenia logowania. Jeśli są poprawne, Django nie zwraca ciasteczek sesyjnych, lecz generuje unikalny **Token** (widoczny w naszej bazie jako TokenProxy). Frontend musi przechwycić ten token i zapisać go u siebie (najlepiej w localStorage lub sessionStorage). Od tego momentu przy **każdym** kolejnym zapytaniu (np. o listę zadań), React musi doklejać ten token do nagłówków HTTP: Authorization: Token <twój_ciag_znakow>.

- **Oczekiwany Payload (JSON):**

JSON

```
{
  "username": "igor22",
  "password": "SuperTajneHaslo123#"
}
```

- **Poprawna Odpowiedź:** Status 200 OK

JSON

```
{
  "token": "9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b"
}
```

2. Następne Wymagane Endpointy

Mając gotową bazę danych i system logowania, musimy teraz stworzyć "okna" na resztę naszego systemu. Zgodnie z naszą architekturą (models.py) i historyjkami użytkownika, w najbliższym czasie będziemy musieli napisać w views.py następujące endpointy:

Moduł Zadań (Priorytet)

Frontend nie ma obecnie czego wyświetlać po zalogowaniu. Musimy wystawić zadania.

- **GET /api/tasks/** – Pobiera listę dostępnych zadań (tylko te z is_public=True oraz zadania z grup, do których należy użytkownik).
- **GET /api/tasks/<id>/** – Zwraca pełne szczegóły pojedynczego zadania (treść Markdown, limity czasowe).
- **POST /api/tasks/** – Pozwala użytkownikom z uprawnieniami (lub pracownikom) dodać nowe zadanie do bazy.

Moduł Rozwiązań (Silnik Platformy)

Użytkownik musi mieć gdzie wysłać swój kod do sprawdzenia.

- **POST /api/solutions/** – Endpoint przyjmujący kod napisany w edytorze na froncie. Zapisuje go do bazy ze statusem pending (Oczekujące) i przygotowuje do weryfikacji.
- **GET /api/solutions/** – Zwraca historię rozwiązań zalogowanego użytkownika (aby mógł zobaczyć, które zadania już zaliczył).

Moduł Społeczności i Grup

Aby działały funkcje socjalne i ograniczenia dostępności.

- **GET /api/groups/** – Zwraca listę otwartych grup (is_open=True), do których można dołączyć.
- **POST /api/groups/<id>/join/** – Endpoint podpinany pod przycisk "Dołącz" na froncie, dodający użytkownika do GroupMembership.

Moduł Konkursów (Opcjonalnie, w późniejszym etapie)

- **GET /api/contests/** – Wyświetlanie aktywnych i nadchodzących konkursów przypisanych do grup użytkownika. To tutaj frontend będzie odczytywał flagę strict_mode, aby aktywować zabezpieczenia antysciągowe w edytorze.